

Advanced Software Engineering

Course Syllabus & Roadmap — Fall 2026 (115-1)

Graduate Seminar (進階軟體工程)

Instructor: Prof. Nien-Lin Hsueh (薛念林 教授)

Department of Information Engineering and Computer Science

Feng Chia University

Course Overview & Philosophy

- **Core Methodologies & The Impact of AI:**

- Master **foundational software engineering methodologies** across the complete lifecycle (specification, architecture, validation, evolution).
- Explore how **Generative & Agentic AI** fundamentally impacts and reshapes every software activity.

- **Graduate-Level Pedagogy: Research & Empirical Practice:**

- **Scholarly Exploration:** Investigate cutting-edge research topics and empirical studies through **paper presentations and panel discussions**.
- **Hands-on Experimentation:** Experience software development directly through **practical implementation and empirical experiments**.

- **Key Course Attributes:**

- **Target Audience:** Graduate students and motivated senior undergraduates.
- **Language of Instruction:** English-friendly / bilingual materials.
- **Interactive Pedagogy:** Concept Check Questions (CCQ) with real-time polling, architectural critiques, and collaborative discussions.

Course Learning Objectives (CLOs)

- **1. Master Core SE Disciplines:**

- Deeply understand software lifecycles, requirement engineering, object-oriented modeling, and architectural styles.

- **2. Quality-First & Empirical Mindset:**

- Internalize testing methodologies (black-box, white-box), CI/CD automation, and empirical experimentation.

- **3. Responsible & Rigorous AI Integration:**

- Learn how to effectively steer AI coding agents, validate AI-generated code, and mitigate hallucinations and technical debt.

- **4. Scholarly & Professional Communication:**

- Deconstruct peer-reviewed SE literature, run conference-style panels, and deliver production-ready software engineering post-mortems.

Curriculum Roadmap: Five Core Modules

- **Module 1: Foundations (Weeks 1)**
 - From the 1968 Software Crisis to Software Engineering; From Software to Qualified Software.
- **Module 2: Processes & Requirements (Weeks 2–4)**
 - Agile Methodologies, Scrum, DevOps, Spec-Driven AI workflows.
 - Requirements Engineering (Functional/Non-Functional Specs, AI Spec)
- **Module 3: Software Design & Modeling (Weeks 5–9)**
 - Design Principles & Patterns, UML Modeling, Software Architectures.
 - Mid-term exam.
- **Module 4: Quality Assurance & Testing (Weeks 10–13)**
 - Testing Fundamentals & Practices: Black-box testing, white-box testing, coverage criteria.
 - Quality Assurance Model & Process Improvement Model
- **Module 5: Empirical Software Engineering & Capstone Execution (Weeks 14–16)**
 - Controlled experiments, final capstone demos and panel debates.

Week 3 (09/28) & Week 7 (10/26): National Holiday

Grading Scheme

- **25% — Midterm Exam**
 - Part I Evaluation (25%)
- **25% — Final Exam**
 - Part II Evaluation (25%)
- **20% — In-Class Engagement & QA**
 - Active questioning during panel and pitches
- **30% — Final Capstone Project**
 - Execution of Track 1, Track 2, or Track 3

Final Capstone Project: Three Flexible Tracks

- **Team-Based Project:** Performed by teams of **~4 members**.
- **Presentation Duration:** Time period = **25 min × number of members** (e.g., ~100 min for 4 members).
- **Track 1: Academic Paper Panel Discussion**
 - Conducted in a **panel discussion style** to present peer-reviewed papers (*ICSE, FSE, ASE, TSE, TOSEM*).
 - The team acts as panelists (~4 persons) deconstructing papers, synthesizing trade-offs, and running Q&A.
- **Track 2: Software System Prototype & SE Post-Mortem**
 - Live demo + in-depth SE post-mortem focusing on a **thin vertical slice**.
 - Architectural separation, AI-assisted workflow logs, CI/CD pipeline, and automated tests.
- **Track 3: Empirical Software Engineering Experiment**
 - Scientific presentation + replication package (dataset & benchmark scripts).
 - Formulate research questions (RQ_1, RQ_2), evaluate tools/models, and analyze threats to validity.

Track 1: Academic Paper Panel — How It Works

- **What is an Academic Paper Panel?**
 - Similar to paper presentations, but with a **unified theme and strong narrative coherence**.
 - Four panelists present four related papers (*ICSE*, *FSE*, *ASE*, *TSE*, *TOSEM*).
 - A **Student Moderator** opens with a theme overview, connects the presentations, provides a final synthesis, and leads audience Q&A.
- **Theme Example: "Security & Trust in Generative AI Code Assistants"**
 - **Moderator Intro:** Why AI code security matters to enterprise systems.
 - **Panelist 1 (*ICSE*):** Prevalence and taxonomy of CWE flaws in Copilot code.
 - **Panelist 2 (*FSE*):** Automated hallucination detection in generated tests.
 - **Panelist 3 (*ASE*):** Developer over-reliance & cognitive bias in AI code reviews.
 - **Panelist 4 (*TOSEM*):** Code licensing contamination and privacy leakages.
 - **Moderator Synthesis & Q&A:** Synthesizing trade-offs and moderating class debate.

Track 1: Academic Paper Panel — Visual Setup & Flow



Track 2: System Prototype & Post-Mortem — Examples

- **System Example: "Smart Food Ordering & Real-Time Delivery Service"**
 - **Thin Vertical Slice:**
 - Customer cart & order placement → Real-time kitchen dispatch webhook → Driver location tracking simulation → Order fulfillment status (secondary billing, reviews, and catalog mocked).
 - **Engineering Rigor:**
 - **Clean Architecture:** Strict separation between Domain entities, Use Cases, and Web/DB adapters.
 - **Event-Driven & Async:** Message queues for order events, schema validation with Pydantic/Zod.
 - **Automated CI/CD & Testing:** Dockerized testing environment with >85% unit and integration test coverage.
 - **SE Post-Mortem Presentation:**
 - Live vertical-slice demo, followed by an in-depth engineering post-mortem:
 - Concurrency bottlenecks, AI coding logs, prompt reproducibility, and architectural trade-offs.

Track 3: Empirical SE Experiment — Examples

- **Study Example A: "Benchmarking Open-Source vs. Commercial Coding LLMs"**
 - **Objective:** Controlled benchmark evaluating code correctness and security across models.
 - **Research Questions:**
 - RQ_1 : Do open-source models (e.g., DeepSeek-Coder, Llama-Code) inject higher rates of CWE flaws than GPT-4o on HumanEval-Security?
 - RQ_2 : How does test-driven iterative prompting affect pass@k accuracy and token costs?
 - **Deliverables:** Curated benchmark datasets, automated evaluation scripts, and replication Docker repo.
- **Study Example B: "Empirical Evaluation of Automated Program Repair (APR) in CI"**
 - **Objective:** Measure real-world patch correctness vs. plausible test-overfitting across APR tools.
 - **Research Questions:**
 - RQ_1 : What percentage of LLM-generated bug fixes are semantically sound vs. merely overfitting?
 - RQ_2 : How does token context window depth impact multi-file bug repair success?
 - **Deliverables:** Defects4J benchmark replication suite, execution telemetry logs, and threat analysis.

Academic Integrity & Generative AI Policy

- **Generative AI is Welcomed, but Must Be Steered Responsibly:**
 - You are encouraged to use AI tools (e.g., Claude, ChatGPT, GitHub Copilot, Cursor).
 - **Rule of Transparency:** Document where and how AI was utilized (prompts, edits, test generation).
- **The "Accountability" Rule:**
 - **You are 100% responsible** for the correctness, security, and architectural integrity of your submissions. Blindly accepting hallucinated code or fabricated paper citations results in severe penalties.
- **Plagiarism & Authorship:**
 - Uncredited copying of text, diagrams, or code from external repositories or human authors is strictly prohibited.

References & Recommended Resources

- **Core References:**

- Ian Sommerville, *Software Engineering*, 10th Edition, Pearson.
- Robert C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*.
- Selected top-tier research papers from *ACM/IEEE ICSE*, *IEEE TSE*, *ACM TOSEM*.

- **Digital Toolchains:**

- **Version Control & CI/CD:** GitHub, GitHub Actions, Docker.
- **Interactive Polling:** NickEduPocket CCQ System ([nickedupocket](#)).
- **Slide & Handout Toolchain:** Marp Markdown Slide Engine.

Course Materials Architecture & Progressive Evolution

- **Three-Pillar Learning Materials Architecture:**

- **1. In-Depth Lecture Notes:** Dual-track Markdown & A4 PDFs ([Lecture/en/](#) & [Lecture/tw/](#)) for theoretical foundations, historical context, and deep dives.
- **2. 16:9 Visual Slide Decks:** Marp-rendered widescreen slides ([Slide/](#)) focusing on core concepts, architectural diagrams, and classroom flow.
- **3. Real-Time Concept Check Questions (CCQs):** Formative micro-assessments with instant QR code mobile polling via [NickEduPocket](#).

- **Human-in-the-Loop AI Collaboration:**

- Selected lecture notes, slide syntheses, and discussion cases are **co-created with LLMs (Gemini & Claude)**.
- **Instructor-Led Rigor:** Curriculum architecture, pedagogical design, and technical verification are strictly led and verified by the instructor.

- **Progressive Updates Throughout the Semester:**

- Course materials are undergoing modern active refinement and **are not fully finalized upfront**.
- Handouts, slides, and challenge problems will be **continuously updated week by week** alongside class pacing. Student feedback and issue reports are warmly welcomed!

Getting Started & Action Items for Week 1

- **Immediate Next Steps:**

1. Review the Three Capstone Tracks (Academic Panel, System Prototype, Empirical Study).
2. Brainstorm topics and start reaching out to potential teammates (teams of ~4).
3. Bring your laptop/mobile device to class for interactive CCQ polls.

- **Course Inquiries & Office Hours:**

- **Instructor:** Prof. Nien-Lin Hsueh (薛念林 教授)
- **Office:** Department of Information Engineering and Computer Science
- **Contact & Appointments:** Via course LMS or email

Questions & Discussion

Welcome to Advanced Software Engineering!

Let's build reliable, elegant, and impactful software systems together.